

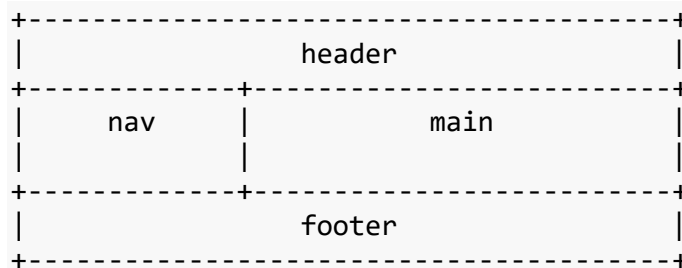
Créer des mises en page (layouts) avec Thymeleaf

1. Principe d'une mise en page

Une **mise en page** (*page layout*) décrit la disposition du texte, des images et des autres éléments sur une page. Un site contient souvent de nombreuses pages qui partagent la même structure : un en-tête, un menu de navigation, une zone de contenu principal et un pied de page. Plutôt que de recopier cette structure dans chaque page, on la définit **une seule fois** dans un gabarit, et chaque page ne fournit que ce qui lui est propre.

Dans Thymeleaf, cette technique repose sur les **fragments** et leurs paramètres : lisez d'abord le document sur les fragments (expressions `~{...}`, `th:replace`, fragments paramétrés).

Structure visée :



À partir de cette structure, on crée plusieurs pages (accueil, produits, à propos) dont l'en-tête, le menu, le contenu et le pied de page diffèrent, mais dont la disposition est identique.

2. Mise en œuvre dans une application Spring Boot

Arborescence

```
src/main
├── java/local/epul4a/demosbt/controller/MainController.java
├── resources
│   ├── static
│   │   ├── main.css
│   │   ├── home.js  home.css  products.js  about.js      (ressources propres
│   │   └── à chaque page)
│   └── templates
│       ├── layouts/main-layout.html
│       ├── fragments/app-fragments.html
│       ├── home.html
│       ├── products.html
│       └── about.html
```

Si vous utilisez un autre framework que Spring Boot, la structure du projet sera légèrement différente, mais le principe reste le même.

Le gabarit : `layouts/main-layout.html`

Le fichier contient **un seul fragment** à six paramètres. Ces paramètres permettent de façonner l'interface complète d'une page :

Paramètre	Rôle
title	Le titre de la page (<title>)
otherStaticResources	Les ressources CSS / JavaScript propres à la page
header	Le contenu de l'en-tête
nav	Le menu de navigation
mainContent	Le contenu principal
footer	Le pied de page

```
<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org"
      th:fragment="main-fragment (title, otherStaticResources, header,
                                nav, mainContent, footer)">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title th:replace="${title}">Titre de la page</title>

  <link rel="stylesheet" href="../../static/main.css" th:href="@{/main.css}"
/>

  <!-- Autres sources CSS / JavaScript propres à la page -->
  <th:block th:replace="${otherStaticResources} ?: ~{}"></th:block>
</head>
<body>
  <header th:insert="${header} ?: ~{}">
    <h2>Titre de la page</h2>
  </header>

  <div class="layout-body">
    <nav th:insert="${nav} ?: ~{}">
      <ul>
        <li><a href="#">Page 1</a></li>
        <li><a href="#">Page 2</a></li>
      </ul>
    </nav>

    <main th:insert="${mainContent} ?: ~{}">
      <h1>Page 1</h1>
      <p>Contenu principal de la page 1</p>
    </main>
  </div>
</body>
</html>
```

```

</div>

<footer th:insert="${footer} ?: ~{}">
  <p>Pied de page</p>
</footer>
</body>
</html>

```

Explications :

- Chaque paramètre contient une **expression de fragment** (`~{...}`) fournie par la page. `th:replace="${title}"` remplace le `<title>` du gabarit par celui de la page ; `th:insert="${header}"` insère le fragment de la page dans la balise `<header>`.
- `${header} ?: ~{}` signifie : « si le paramètre est absent (`null`), insérer le fragment vide `~{}` », c'est-à-dire rien. Le gabarit ne plante donc pas si une page n'a pas de menu, par exemple.
- Le contenu écrit à l'intérieur des balises (`<h2>Titre de la page</h2>`, etc.) sert de **maquette** : il s'affiche quand on ouvre `main-layout.html` directement dans un navigateur (*Natural Template*), mais n'apparaît pas dans les pages générées.
- Le gabarit utilise les éléments sémantiques HTML5 `<header>`, `<nav>`, `<main>` et `<footer>`, plus adaptés que des `<article>` ou des `<div>` génériques.
- La balise `<meta name="viewport" ...>` est indispensable pour que la règle CSS adaptative (`@media`) fonctionne sur téléphone.

Variante : conserver le contenu du gabarit par défaut. Avec

`th:insert="${header} ?: _"` (le jeton `_`, *no-operation*), c'est le contenu de la maquette écrit dans le gabarit qui sert de **valeur par défaut** quand la page ne fournit rien. Avec `~{}`, la zone reste vide.

La feuille de style : `static/main.css`

La disposition en colonnes utilise **Flexbox**, plus simple et plus robuste que les anciens float avec « clearfix ».

```

*, *::before, *::after { box-sizing: border-box; }

body { margin: 0; font-family: system-ui, sans-serif; }

header {
  background-color: #666;
  padding: 5px;
  text-align: center;
  font-size: 35px;
  color: white;
}

.layout-body { display: flex; min-height: 300px; }

nav {

```

```
flex: 0 0 30%;
background: #ccc;
padding: 20px;
}
nav ul { list-style-type: none; padding: 0; }

main {
  flex: 1;
  padding: 20px;
  background-color: #f1f1f1;
}

footer {
  background-color: #777;
  padding: 10px;
  text-align: center;
  color: white;
}

@media (max-width: 600px) {
  .layout-body { flex-direction: column; }
  nav { flex-basis: auto; }
}
```

Ouvrez `main-layout.html` directement dans le navigateur : vous voyez l'interface de la maquette.

Les fragments communs : `fragments/app-fragments.html`

Ce fichier regroupe des fragments réutilisés par plusieurs pages.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8" />
  <title>Fragments de l'application</title>
</head>
<body>
  <!-- Menu par défaut -->
  <ul th:fragment="nav-default">
    <li><a th:href="@{/}">Accueil</a></li>
    <li><a th:href="@{/products}">Produits</a></li>
    <li><a th:href="@{/about}">À propos</a></li>
  </ul>

  <!-- Menu d'administration -->
  <ul th:fragment="nav-admin">
    <li><a th:href="@{/admin/products}">Gestion des produits</a></li>
    <li><a th:href="@{/admin/orders}">Gestion des commandes</a></li>
  </ul>
```

```
<!-- Pied de page : l'année est calculée, pas écrite en dur -->
<div th:fragment="copyright">
    &copy; Mon site -
    <span th:text="${#temporals.year(#temporals.createNow())}">2026</span>
</div>
</body>
</html>
```

Les pages

Chaque page remplace sa balise `<html>` par le gabarit, en lui passant des expressions de fragment qui désignent ses propres éléments.

home.html :

```
<!DOCTYPE html>
<!-- main-fragment (title, otherStaticResources, header, nav, mainContent,
footer) -->
<html xmlns:th="http://www.thymeleaf.org"
      th:replace="~{layouts/main-layout :: main-fragment(
          ~{::title},
          ~{:: #home-static-resources},
          ~{:: #home-header},
          ~{:: #home-nav},
          ~{:: #home-main-content},
          ~{:: #home-footer}
      )}">
<head>
  <title>Titre de la page d'accueil</title>

  <th:block id="home-static-resources">
    <script src="../../static/home.js" th:src="@{/home.js}" defer></script>
    <link rel="stylesheet" href="../../static/home.css" th:href="@{/home.css}"
  />
  </th:block>
</head>
<body>
  <div id="home-header">
    En-tête de la page d'accueil
  </div>

  <div id="home-nav" th:replace="~{fragments/app-fragments :: nav-default}">
    Menu
  </div>

  <div id="home-main-content">
    <h2>Accueil</h2>
    <div>Contenu de la page d'accueil</div>
  </div>
</body>
</html>
```

```
<div id="home-footer" th:replace=~{fragments/app-fragments :: copyright}>
  Pied de page
</div>
</body>
</html>
```

products.html suit exactement la même structure : seuls les identifiants (products-header, products-nav, products-main-content, products-footer, products-static-resources), le titre et le contenu changent.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      th:replace=~{layouts/main-layout :: main-fragment(
        ~{::title},
        ~{:: #products-static-resources},
        ~{:: #products-header},
        ~{:: #products-nav},
        ~{:: #products-main-content},
        ~{:: #products-footer}
      )}>
<head>
  <title>Titre de la page Produits</title>

  <th:block id="products-static-resources">
    <script src="../../static/products.js" th:src="@{/products.js}"
defer></script>
  </th:block>
</head>
<body>
  <div id="products-header">En-tête de la page Produits</div>

  <div id="products-nav" th:replace=~{fragments/app-fragments :: nav-
default}>Menu</div>

  <div id="products-main-content">
    <h2>Produits</h2>
    <p>Samsung</p>
    <p>iPhone</p>
    <p>Nokia</p>
  </div>

  <div id="products-footer" th:replace=~{fragments/app-fragments ::
copyright}>
    Pied de page
  </div>
</body>
</html>
```

about.html est construite de la même façon, avec le préfixe about-.

Points importants :

- **Cohérence des identifiants.** Chaque identifiant cité dans `~{:: #home-header}` doit exister, à l'identique, dans la page (`id="home-header"`). Une faute d'orthographe ou un tiret manquant donne une zone vide.
- **Pas de / initial** dans les chemins : `~{fragments/app-fragments :: nav-default}`.
- **Tous les paramètres sont obligatoires.** Le fragment `main-fragment` en déclare six : une page qui n'a pas de ressources propres doit tout de même en passer six, avec `~{}` pour celles qu'elle n'utilise pas : `main-fragment(~{::title}, ~{}, ~{:: #about-header}, ...)`.
- **Le reste de la page est ignoré.** Comme la balise `<html>` de la page est remplacée, seules les parties sélectionnées sont reprises dans le résultat. Le contenu des autres balises sert uniquement à afficher la maquette dans un navigateur.
- Les éléments sélectionnés peuvent eux-mêmes utiliser Thymeleaf : dans `home.html`, `<div id="home-nav" th:replace=...>` est traité une fois inséré dans le gabarit, et son contenu est remplacé par le fragment de menu.

Le contrôleur

```
package local.epul4a.demosbt.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class MainController {

    @GetMapping("/")
    public String home() { return "home"; }

    @GetMapping("/products")
    public String products() { return "products"; }

    @GetMapping("/about")
    public String about() { return "about"; }
}
```

3. Simplifier : un gabarit qui fournit des valeurs par défaut

Six paramètres obligatoires dans chaque page, c'est lourd. Dans la plupart des sites, le menu et le pied de page sont les **mêmes partout** : il est plus simple de les placer directement dans le gabarit, et de ne demander aux pages que ce qui change.

```
<html lang="fr" xmlns:th="http://www.thymeleaf.org"
      th:fragment="layout (title, content, extraResources)">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
```

```

<title th:replace="${title}">Titre</title>
<link rel="stylesheet" th:href="@{/main.css}" />
<th:block th:replace="${extraResources} ?: ~{}"></th:block>
</head>
<body>
  <header><h2>Mon site</h2></header>
  <div class="layout-body">
    <nav th:insert=~{fragments/app-fragments :: nav-default}></nav>
    <main th:replace="${content}"></main>
  </div>
  <footer th:insert=~{fragments/app-fragments :: copyright}></footer>
</body>
</html>

```

Une page n'écrit plus que :

```

<html xmlns:th="http://www.thymeleaf.org"
      th:replace=~{layouts/simple-layout :: layout(~{::title}, ~{::main},
~{::})}>
<head><title>Produits</title></head>
<body>
  <main>
    <h1>Produits</h1>
    <p>Contenu de la page.</p>
  </main>
</body>
</html>

```

Notez l'emploi de `th:insert` pour `<nav>` et `<footer>` : le fragment est placé **dans** la balise du gabarit, qui reste présente et conserve son style (avec `th:replace`, la balise serait remplacée par le fragment). La structure commune est ainsi modifiée à un seul endroit, et chaque page se réduit à son titre et à son contenu.

4. Une autre approche : Thymeleaf Layout Dialect

La bibliothèque tierce **Thymeleaf Layout Dialect** propose une approche par **décoration** de gabarit, proche de l'héritage. Elle est indépendante de Thymeleaf. Avec Spring Boot, il suffit d'ajouter la dépendance `nz.net.ultraq.thymeleaf:thymeleaf-layout-dialect` (sa version est gérée par Spring Boot), et le dialecte est configuré automatiquement.

Gabarit :

```

<html xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      xmlns:th="http://www.thymeleaf.org">
<head>
  <title layout:title-pattern="$CONTENT_TITLE - Mon site">Mon site</title>
  <link rel="stylesheet" th:href="@{/main.css}" />
</head>
<body>

```



```
<header><h2>Mon site</h2></header>
<main layout:fragment="content">Contenu par défaut</main>
<footer>&copy; Mon site</footer>
</body>
</html>
```

Page :

```
<html layout:decorate="~{layouts/main-layout}"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">
<head><title>Produits</title></head>
<body>
  <main layout:fragment="content">
    <h1>Produits</h1>
  </main>
</body>
</html>
```

	Fragments paramétrés (Thymeleaf natif)	Layout Dialect
Dépendance	Aucune	Bibliothèque tierce
Principe	La page appelle le gabarit avec des paramètres	La page « hérite » du gabarit et redéfinit des zones nommées
Titre composé	À gérer soi-même	layout:title-pattern
Maintenance	Suit les versions de Thymeleaf	Dépend du projet tiers : vérifier la compatibilité avec votre version de Thymeleaf et de Spring Boot

Les deux approches sont valables. Pour un petit projet, les fragments natifs suffisent largement ; vérifiez la compatibilité de version avant d'ajouter le dialecte.

5. Erreurs fréquentes

Symptôme	Cause probable
Une zone du gabarit reste vide	L'identifiant passé (~{ : : #home-header }) ne correspond pas à un id de la page (faute de frappe, tiret manquant)
Exception « Cannot resolve fragment... parameters »	Le nombre d'arguments de l'appel ne correspond pas à la liste de paramètres du fragment
Template introuvable	Chemin avec / initial, extension .html écrite, ou fichier mal placé dans templates/
La mise en page « casse » sur téléphone	<meta name="viewport"> absent
Le contenu de la page n'apparaît pas	L'élément n'est pas sélectionné par un paramètre : seul ce qui est passé au gabarit est repris

Résumé

- Une mise en page se construit avec un gabarit (un fragment paramétré) que chaque page appelle avec `th:replace` sur sa balise `<html>`.
- Les paramètres sont des expressions de fragment (`~{::title}`, `~{:: #id}`) qui désignent les éléments de la page courante.
- `${param} ? : ~{ }` rend un paramètre facultatif ; `? : _` conserve le contenu du gabarit par défaut.
- Les fragments communs (menu, pied de page) se placent dans un fichier dédié ; un gabarit qui les intègre directement allège les pages.
- Thymeleaf Layout Dialect est une alternative tierce par décoration de gabarit.
- Les identifiants cités dans les expressions doivent correspondre exactement à ceux de la page.

Exercices proposés

1. Ajouter à l'application une page « Contact » qui utilise le gabarit à six paramètres, sans ressources statiques propres (utiliser `~{ }`).
2. Faire afficher, selon que l'utilisateur est administrateur ou non, le fragment `nav-admin` ou `nav-default` dans le menu (expression de fragment dans une condition).
3. Mettre en évidence le lien du menu correspondant à la page courante (voir l'exemple `currentUri` avec `@ControllerAdvice` du document sur les objets prédéfinis).
4. Réécrire les trois pages avec le gabarit simplifié de la section 3, et comparer le nombre de lignes avec la version à six paramètres.
5. Réaliser la même application avec Thymeleaf Layout Dialect et comparer les deux approches (lisibilité, dépendances, messages d'erreur).